Spotting Big O in Code

Learn to read code and know the runtime instantly

Intro to CS | Runtime Analysis

O(n) — Linear Time

👍 Fast enough

One loop through the data = time grows with input size.

```python
# Looping through a list
nums = [1, 2, 3]
for n in nums:
    print(n)
```

← one loop = O(n)

```python
# Also O(n) — built-in ops
sum(nums)          # sum of list
nums.insert(1, 100)  # insert mid
nums.remove(100)   # remove mid
print(100 in nums)  # search
```

The Pattern

See ONE loop? Or a built-in that scans the whole list? → O(n). If the list doubles, the work doubles.

10 items → 10 steps | 1,000 items → 1,000 steps | 1M items → 1M steps

O(n²) — Quadratic Time

Gets slow fast

A loop inside a loop = time explodes as input grows.

```python
# Traverse a square grid
nums = [[1,2,3],[4,5,6]]
for i in range(len(nums)):
  for j in range(len(nums[i])):
    print(nums[i][j])
```

← nested = O(n²)

```python
# Get every pair of elements
nums = [1, 2, 3]
for i in range(len(nums)):
  for j in range(i+1, len(nums)):
    print(nums[i], nums[j])
# Outputs: (1,2) (1,3) (2,3)
```

The Pattern

See a LOOP INSIDE A LOOP? → O(n²). If the list doubles, the work quadruples. Bubble sort, insertion sort = O(n²).

10 items → 100 steps | 1,000 items → 1,000,000 steps | Avoid for large data!

O(log n) — Logarithmic

🚀 Super fast

Cut the problem in half each step — the hallmark of binary search.

```
# Binary Search
nums = [1, 2, 3, 4, 5]
target = 4
l, r = 0, len(nums) - 1

while l <= r:
    m = (l + r) // 2
    if target < nums[m]:
        r = m - 1
    elif target > nums[m]:
        l = m + 1
    else:
        print(m)
        break
```

← halves the search space each step

The Trick

Each iteration removes HALF the remaining data. That's what makes it O(log n).

⚠ **Requires sorted data!**

How to Spot It

`while` with `l = mid + 1`

Dividing search range = O(log n)

1,000,000 items → ~20 steps | 1 Billion items → ~30 steps | This is why binary search is so powerful.

Cheat Sheet: Spot the Runtime

Big O	Code Pattern	Example
O(1)	No loops. Direct access.	`nums[5], len(x)`
O(log n)	Loop that halves the input.	`Binary search`
O(n)	One loop through all items.	`for n in nums:`
O(n log n)	Sort, then use the result.	`sorted(), .sort()`
$O(n^2)$	Loop inside a loop.	`for i: for j:`
$O(2^n)$	Every combination / subset.	`Brute-force recursion`

Quick rule: count your loops. 0 loops = O(1). 1 loop = O(n). 2 nested loops = $O(n^2)$. Halving = O(log n).